

Séance 5 – Héritage

Université Côte d'Azur – LJAD

La composition : « possède un »

Une classe peut avoir comme membres d'autres objets.

```
class Mesh {  
public:  
    std::size_t node_count() const;  
  
private:  
    std::vector<Node> nodes_  
    std::vector<Cell> cells_  
};
```

- Les membres sont des parties de l'objet Mesh.
- Leur durée de vie suit celle du maillage.

Composition

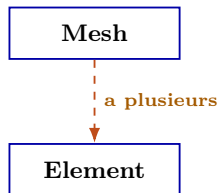
« possède un »

Mesh possède notamment
`std::vector<Triangle>`.

La composition décrit de quelles parties un objet est fait.

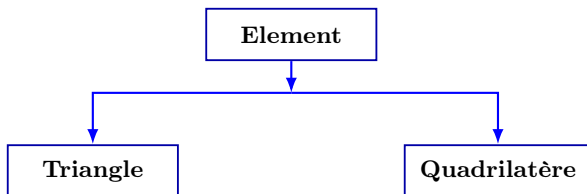
Une seconde relation : « est-un »

Composition



```
class Mesh {  
    std::vector<Element> elements_;  
};
```

Héritage



```
class Triangle : public Element {  
    // informations supplémentaires  
};
```

Deux questions différentes : « possède-t-il ? » et « peut-il être utilisé comme ? ». Aucune relation ne remplace l'autre.

Composition et héritage, côte à côte

Composition : x possède un membre de type y .

```
class Trajectory {  
private:  
    std::vector<TimedState> states_;  
};
```

À lire : « une trajectoire possède des états »

L'interface de Trajectory pourra imposer des temps croissants.

À retenir. La différence syntaxique est petite ; l'engagement sémantique de l'héritage est beaucoup plus fort.

Héritage : x annonce qu'il est un y spécialisé.

```
class Triangle : public Element {  
private:  
    std::array<Node, 3> nodes_;  
};
```

À lire : « un triangle est un élément »

Le

code qui attend un Element doit pouvoir recevoir un Triangle.

Le test qui tranche : puis-je remplacer la base ?

Une trajectoire est-elle un

`std::vector<TimedState>` ?

`std : :vector<TimedState>`

Trajectory

✗ Un client du `vector` peut insérer un état dans le désordre : les temps ne sont plus croissants.

Un triangle est-il un `Element` ?

Element

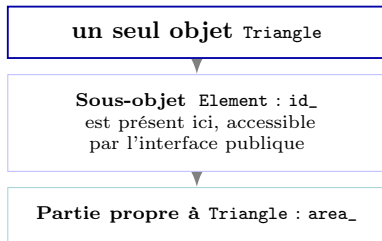
Triangle

✓ Il respecte l'identité et les opérations communes prévues pour un élément.

Test pratique : le type dérivé respecte-t-il tout le contrat observable de la base ?

Ce qui existe réellement dans un objet dérivé

```
class Element {  
public:  
    explicit Element(int id) : id_{id} {}  
    int id() const { return id_; }  
  
private:  
    int id_;           // present, mais prive  
};  
  
class Triangle : public Element {  
public:  
    Triangle(int id, double area)  
        : Element{id}, area_{area} {}  
  
private:  
    double area_;  
};
```



Les membres privés de la base restent présents, mais sont inaccessibles directement depuis la dérivée.

Sous-objet de base + partie dérivée = un objet complet

Construire un objet dérivé

Le code et son résultat

```
Triangle::Triangle(int id,  
                  double area)  
    : Element{id}, area_{area} {}
```

La liste d'initialisation construit explicitement la partie `Element`.

Exécution et résultat

```
Triangle t{7, 2.5};  
std::cout << t.id() << '\\n';
```

Sortie :
Element()
Triangle()
7

La base est construite avant la partie propre à `Triangle`.

Détruire dans l'ordre inverse

Le code du destructeur

```
Triangle::~~Triangle() {  
    std::cout << "~Triangle\\n";  
}  
  
Element::~~Element() {  
    std::cout << "~Element\\n";  
}
```

Chaque classe libère ses propres ressources dans son destructeur.

Exécution et résultat

```
{  
    Triangle t{7, 2.5};  
}  
  
Sortie :  
~Triangle  
~Element
```

À la sortie du bloc, la dérivée est détruite avant sa base.

Chaque sous-objet gère sa propre durée de vie : c'est encore le RAII.

Héritage public et contrat de la classe

```
class Element {
public:
    explicit Element(int id) : id_{id} {}
    int id() const { return id_; }
private: int id_;
};

class Triangle : public Element {
    // Triangle est un Element
public:
    Triangle(int id, double area) : Element{id},
        area_{area} {}
private: double area_;
};

void inspect(const Element& e) {
    // inspect ne connaît que l'interface Element
    std::cout << "element #" << e.id();
}

Triangle t{7, 2.5};
inspect(t);    // on lui passe pourtant un Triangle
// sortie : element #7
```

Fonction définie pour

Element

↑ peut être transmis à

Triangle

respecte le contrat de la base

inspect() ne dépend que du

contrat public de Element.

Triangle peut enrichir l'objet avec
area_, sans modifier ce contrat.

Hériter publiquement :
le dérivé respecte le
contrat de la base.

Mais le compilateur peut-il vérifier que ce contrat a du sens ?

Quand la syntaxe ment : Carré / Rectangle

Le code reste syntaxiquement valide

```
class Rectangle {
public:
    void set_width(double w);
    void set_height(double h);
};

class Square : public Rectangle {
    // Square est un Rectangle ?
};

Square s;
s.set_width(4.0);
s.set_height(2.0); // quelle règle du contrat est
                   rompue ?
```

Le contrat de la classe

largeur = hauteur

doit rester vraie après chaque opération

Le rectangle autorise deux

modifications indépendantes. Le carré, lui, doit conserver son invariant.

La syntaxe accepte
l'héritage ; elle
ne vérifie pas le
sens de la relation.

Le compilateur vérifie la syntaxe, pas la validité du contrat « est-un ».

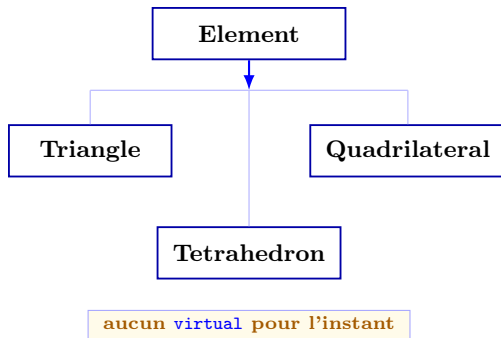
Une famille d'éléments de maillage

```
class Element {
public:
    explicit Element(int id) : id_{id}
    {}
    int id() const { return id_; }

private:
    int id_;
};

class Triangle : public Element {
    std::array<Node, 3> nodes_;
};

class Quadrilateral : public Element {
    std::array<Node, 4> nodes_;
};
```



La géométrie sert d'illustration : le sujet informatique reste la relation entre types et leur manipulation.

Deux façons d'utiliser un *Triangle* comme *Element*

```
Triangle triangle{7, 2.5};

const Element& ref = triangle; // même objet
Element copy = triangle;       // nouvel objet

std::cout << triangle.id() << '\\n'; // 7
std::cout << ref.id() << '\\n';      // 7
std::cout << copy.id() << '\\n';     // 7
std::cout << triangle.area() << '\\n'; // 2.5
```

triangle

Element + par-
tie Triangle

ref → **même objet**

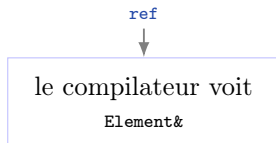
copy → **nouvel** Element

ref et copy représentent-ils la même chose ?

Les deux lignes compilent, mais elles n'ont pas le même effet.

Le type statique : ce que le compilateur voit

```
Triangle triangle{7, 2.5};  
const Element& ref = triangle;  
//      ~~~~~  
// type statique : Element  
  
ref.id();      // OK : id() existe dans Element  
  
// ref.area(); // erreur de compilation :  
//           // area() n'existe pas dans  
//           Element
```



opérations accessibles
interface de Element

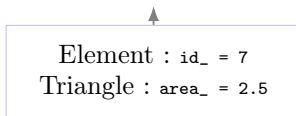
Le compilateur vérifie le code à partir du contrat du type statique, ici `Element`.

Mais quel objet `ref` désigne-t-elle réellement ?

Le type dynamique : l'objet réellement désigné

```
Triangle triangle{7, 2.5};  
const Element& ref = triangle;  
// type statique : Element  
// objet réel    : Triangle  
  
const Element* ptr = &triangle;  
// type statique : Element*  
// objet désigné : Triangle
```

référence ou pointeur de base



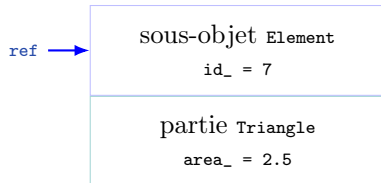
Une référence ou un pointeur de base peut désigner un objet dérivé complet.

Le type statique décrit l'accès (`Element&`); le type dynamique décrit l'objet réellement désigné (`Triangle`). L'objet dérivé reste complet.

Pour l'instant, cette information ne change encore aucun appel de fonction.

Une référence ne fabrique pas un nouvel objet

```
Triangle triangle{7, 2.5};  
  
const Element& ref = triangle;  
std::cout << triangle.id() << '\\n'; // 7  
std::cout << ref.id() << '\\n';      // 7
```

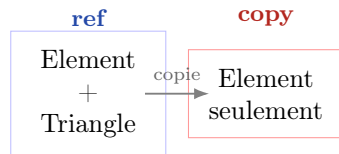


Un seul objet `Triangle` existe en mémoire.

Une référence fournit un autre accès au même objet : elle ne construit rien et conserve la partie propre à `Triangle`.

Copier vers la base : le slicing

```
Triangle triangle{7, 2.5};  
const Element& ref = triangle; // même objet  
Element copy = triangle;      // nouvel Element  
  
std::cout << ref.id() << '\\n'; // 7  
std::cout << copy.id() << '\\n'; // 7  
// copy.area();                // impossible
```



La partie propre à `Triangle` n'est pas copiée : c'est le **slicing**.

Une référence de base conserve l'objet dérivé ; une copie vers la base crée un nouvel objet de base.


```
class Element {  
public:  
    int id() const { return id_; }  
private:  
    int id_;  
};  
  
class Triangle : public Element {  
    std::array<Node, 3> nodes_;  
};  
  
void print_id(const Element& e) {  
    std::cout << e.id() << '\\n';  
}
```

1. Relation entre classes

Un Triangle respecte-t-il
le contrat de Element ?

2. Variation du comportement

L'opération doit-elle dépendre
du type réel de l'objet ?

La relation « est-un » et le choix d'un comportement selon le type réel sont deux questions distinctes.

Démo : l'appel non virtuel suit le type statique

```
#include <iostream>
#include <string>

struct Element {
    std::string label() const { return "Element"; }
};

struct Triangle : Element {
    std::string label() const { return "Triangle"; }
};

void print(const Element& e) {
    std::cout << e.label() << '\n';
}

int main() {
    Triangle t;
    print(t);
}
```

Prédiction

Quel texte sera affiché par
print(t) ?

Résultat

Element

Dans print, la variable `e` a pour type statique `Element&`. Le compilateur sélectionne donc `Element::label()`.

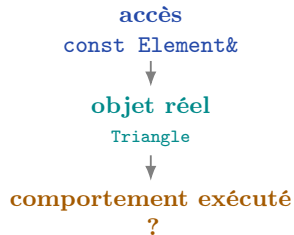
Ce que nous savons désormais

Relation entre classes	L'héritage public exprime une relation « est-un » : le dérivé doit respecter le contrat de la base.
Deux types	Le type statique vient de la déclaration ; le type dynamique décrit l'objet réellement désigné.
Deux effets	Une référence conserve l'objet dérivé complet ; une copie par valeur vers la base produit du slicing.

À ce stade, un appel non virtuel reste déterminé par le type statique.

Question laissée ouverte

```
Triangle triangle{7, 2.5};  
const Element& ref = triangle;  
  
// type statique : Element  
// objet réel    : Triangle  
  
ref.label(); // quel comportement choisir ?
```



Comment sélectionner un comportement
à partir du type réel, sans copier l'objet ?

Cette question introduit la séance suivante : interfaces abstraites et polymorphisme dynamique.